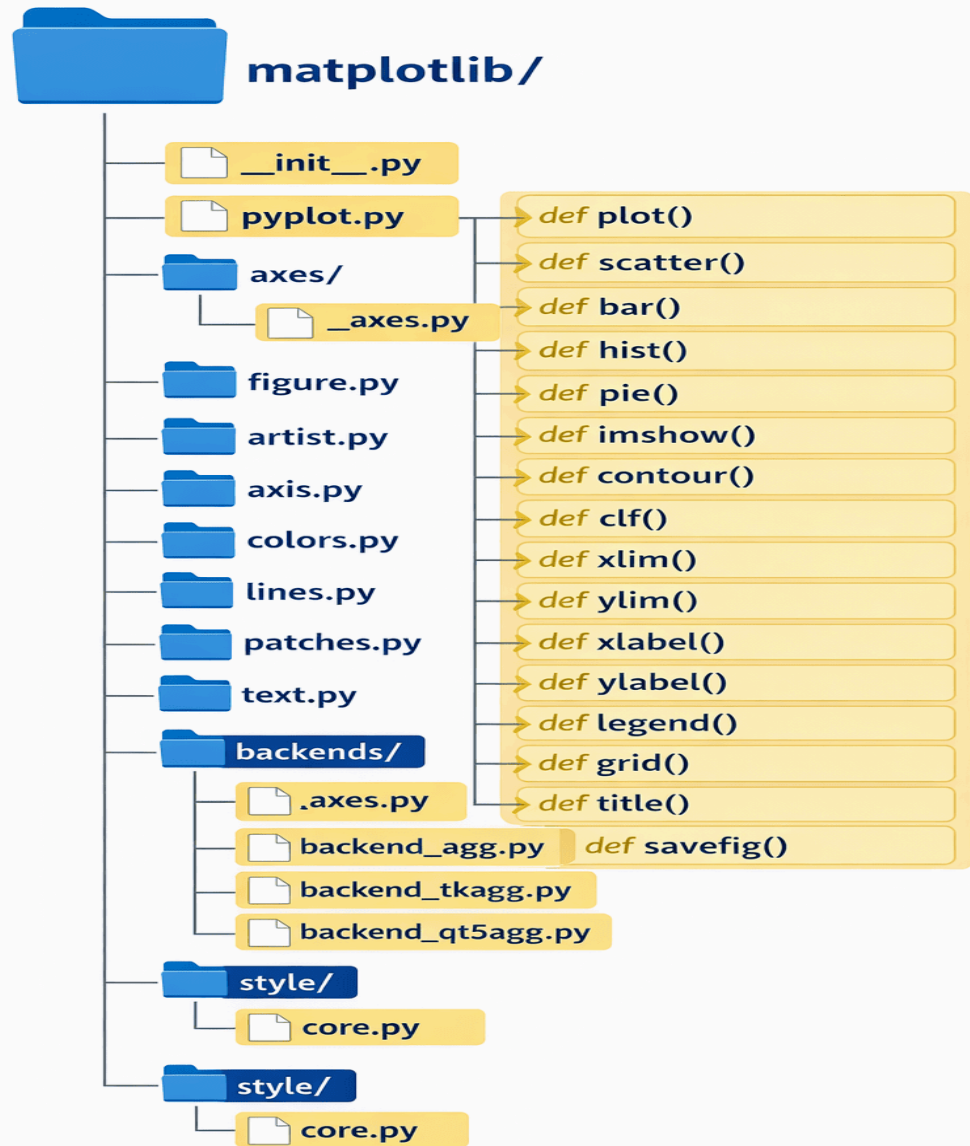


Matplotlib Folder Structure



Line Plot

A **Line Plot** is a type of chart used to show **changes or trends over time** by connecting data points with straight lines.

It is commonly used for:

- Temperature change over days
- Sales growth over months
- Student performance over exams
- Population growth over years

In Python, line plots are usually created using **Matplotlib** and data is often handled using **Pandas**.

Necessary Steps to Create a Line Plot

Step 1 – Import Necessary Things

First, import the required libraries.

```
import pandas as pd
import numpy as np      ← Import if needed .
import matplotlib.pyplot as plt    or from matplotlib import
pyplot as plt
```

- **pandas** → used to create and manage data
 - **matplotlib** → used to create graphs
 - **numpy** → used for numerical calculation.
-

Step 2 – Create Data if Necessary.

Create data using a dictionary.

```
data = {  
    "Day": [1, 2, 3, 4, 5],  
    "Sales_A": [100, 120, 140, 160, 180],  
    "Sales_B": [90, 110, 150, 170, 200]  
}
```

Step 3 – Create DataFrame

OPTION1 - Convert the dictionary into a DataFrame.

```
df = pd.DataFrame(data)
```

OPTION2 - Get DataFrame by reading csv /json/excel file using respective functions .

```
df = pd.read_csv(CSV FILE PATH)   OR  
  
df = pd.read_json(JSON FILE PATH) OR  
  
df = pd.read_excel(EXCEL FILE PATH)
```

NOTE : read_csv() , read_json(), read_excel() function always return a DataFrame .

Step 4 – Define X and Y

```
X = df["Day"]  
Y_StoreA = df["Sales_A"]  
  
Y_StoreB = df["Sales_B"]
```

Step 5 – Create Single/Multi-Line Plot

Formula:

```
plt.plot(X, Y, color = "value", linewidth=LineWidthvalue, marker= MarkerValue,  
markerfacecolor = "value", markeredgewidth=WidthValue, label=LabelValue, )
```

X, Y → Compulsory ; others(linewidth, color etc) are optional

X, Y → Positional argument

Othesr → Keyword arguments

Entities in formula:

```
plt → module (matplotlib.pyplot)  
plot → Function used to create a line plot.  
X → Independent variable (horizontal axis) [cause / input]  
Y → Dependent variable (vertical axis)[effect / output]
```

For line :

color → Color of a line.

linewidth → controls the **thickness of the line** in a plot. (0 to 5) [Default value=1.5]

For Legend :

label → used for legend

For Marker :

marker → shows **points on the line** ['o', 's', '^', 'v', '*', '+', 'x', 'D', 'l', 'p']

[Click here to see Marker table from matplotlib documentation](#)

markeredgecolor → controls the **color of the border (edge) of the marker**.

[Click here to see List of named color from matplotlib.](#)

markeredgewidth → controls the **thickness of the marker border**.(0-5) Default : 1.5

markerfacecolor → Color of a marker

Example:

```
plt.plot(X, Y_StoreA, marker='o', markeredgecolor="black",  
label="Store A")  
plt.plot(X, Y_StoreB, marker='s', markeredgewidth=2,  
label="Store B")
```

You can skip Step4 and write the following code to create line plot.

```
plt.plot(df["Day"], df["Sales_A"], marker='o', label="Store A")  
plt.plot(df["Day"], df["Sales_B"], marker='s', label="Store B")
```

Step 6 — Add X-axis Label by using xlabel function

```
plt.xlabel("Days", fontsize=14, color="blue", fontweight="bold")
```

Step 7 – Add Y-axis Label by using ylabel function

```
plt.ylabel("Sales", fontsize=14, color="green", fontstyle="italic")
```

Step 8 – Add Title by using title function

```
plt.title("Sales Comparison Between Two Stores", fontsize=18,  
color="black")
```

Step 9 – Add Legend by using legend function

```
plt.legend()
```

Legend shows which line belongs to which dataset.

NOTE : To use `legend()`, you must specify `label=LabelValue` inside the `plot()`

function.

Step 10 – Display Plot using show function .

```
plt.show()
```

Complete Example

```
# Step 1 - Import Necessary Things
import pandas as pd
import matplotlib.pyplot as plt

# Step 2 - Create Data if needed
data = {
    "Day": [1, 2, 3, 4, 5],
    "Sales_A": [100, 120, 140, 160, 180],
    "Sales_B": [90, 110, 150, 170, 200]
}

# Step 3 - Create DataFrame
df = pd.DataFrame(data)

# Step 4 - Define X and Y
X = df["Day"]
Y_StoreA = df["Sales_A"]
Y_StoreB = df["Sales_B"]
```

```
# Step 5 - Create Line Plot
plt.plot(
    X,
    Y_StoreA,
    marker='o',
    label="Store A",
    linewidth=2,
    markeredgecolor="black",
    markeredgewidth=2
)
```

```
plt.plot(
    X,
    Y_StoreB,
    marker='s',
    label="Store B",
    linewidth=2,
    markerfacecolor="blue",
    markeredgewidth=2
)
```

```
# Step 6 - Add X-axis Label
plt.xlabel(
    "Days",
    fontsize=14,
    color="blue",
    fontweight="bold"
)
```

```
# Step 7 - Add Y-axis Label
plt.ylabel(
    "Sales",
    fontsize=14,
    color="green",
    fontstyle="italic"
)
```

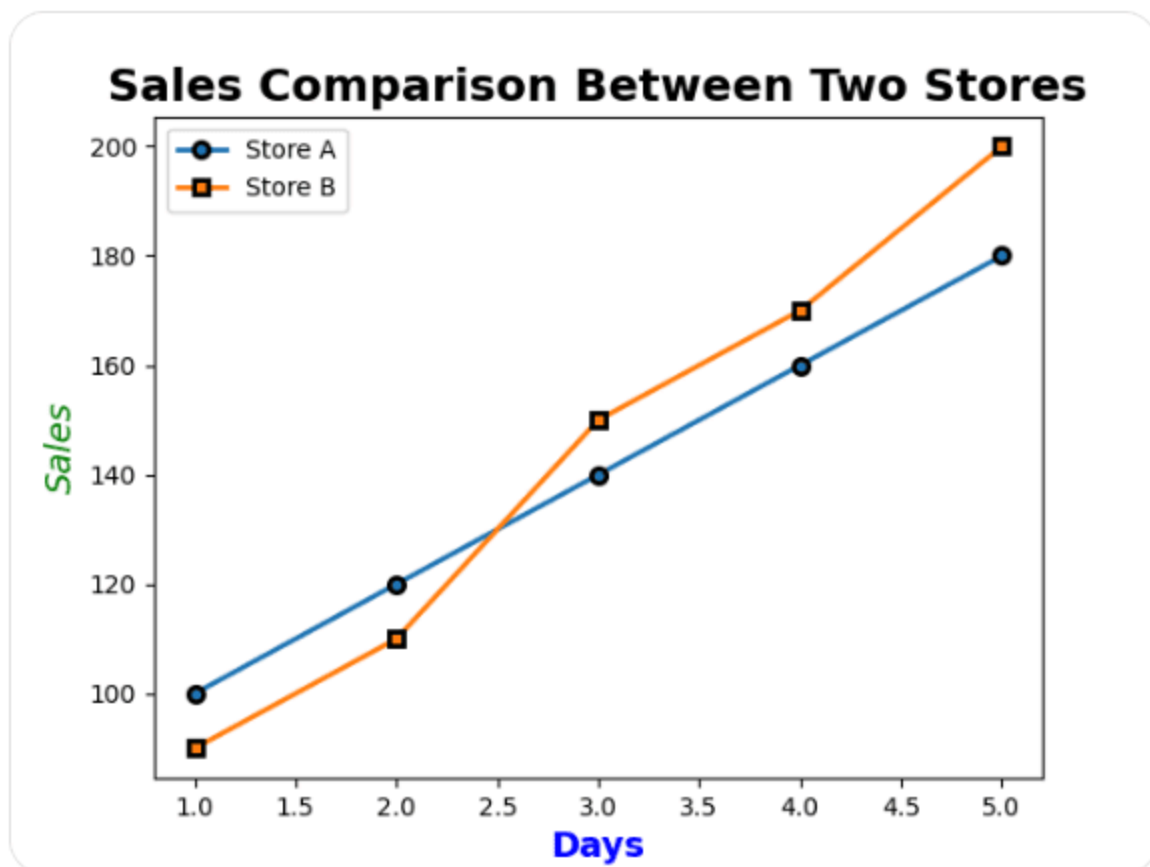
```
# Step 8 - Add Title
plt.title(
```



```
"Sales Comparison Between Two Stores",  
fontsize=18,  
color="black",  
fontweight="bold",  
loc="center"
```

```
)  
  
# Step 9 - Add Legend  
plt.legend()
```

```
# Step 10 - Display Plot  
plt.show()
```





The graph is showing:

- **X-axis** → Days
 - **Y-axis** → Sales
 - **Line 1** → Store A Sales
 - **Line 2** → Store B Sales
 - **Markers** → highlight each data point
 - **Legend** → identifies each line
-

ADDITIONAL :

1 - Show values on the plot by using text function.

```
# plt.text(x_position, y_position, value_to_display)
```

```
# used to display text at a specific position on a plot.
```

```
for i, value in enumerate(df["Sales_A"]):  
    plt.text(df["Day"][i], value, value)  
  
for i, value in enumerate(df["Sales_B"]):  
    plt.text(df["Day"][i], value, value)
```

2 - Show grid by using grid() function.

```
plt.grid()
```